

System symulacji quadcoptera

ROS 2 Humble · Gazebo Classic · RViz 2

Architektura, sterowanie, model wirników i wizualizacja

Dokumentacja pakietu quadcopter_sim
28 sierpnia 2026

Spis treści

- 1 Cel i zakres symulacji
- 2 Architektura całego systemu
- 3 Świat Gazebo i model quadcoptera
- 4 Misja i generator trajektorii
- 5 Regulatory PID, LQR i MPC
- 6 Alokacja sterowania na cztery wirniki
- 7 Model silników i wirników
- 8 Pełna pętla sprzężenia zwrotnego
- 9 Gazebo, RViz i porównanie trajektorii
- 10 Uruchamianie i diagnostyka
- 11 Założenia i ograniczenia modelu

1. Cel i zakres symulacji

Pakiet quadcopter_sim przedstawia quadcopter wykonujący zadaną misję w środowisku Gazebo. Dron otrzymuje płynnie zmieniającą się pozycję zadaną, porównuje ją ze swoim aktualnym stanem i dobiera prędkości czterech wirników. Gazebo oblicza wynikający z nich ruch bryły sztywnej, grawitację oraz kolizje z przeszkodami.

Symulacja pozwala porównywać trzy metody regulacji: kaskadowy PID, LQR oraz uproszczony MPC. Dostępne są dwa scenariusze: pierwotny świat oraz rozbudowany tor przeszkód z murem, slalomem, bramą i lądowiskiem.

Najważniejsza własność: sterowanie wykonawcze nie jest już idealną siłą przykładaną bezpośrednio przez regulator. Wymagany ciąg i momenty są rozdzielane na cztery zadane prędkości wirników, a silniki osiągają je z określoną dynamiką.

2. Architektura całego systemu

```
mission_*.yaml ↓ trajectory_node.py → /quadcopter/command/pose ↓ PID / LQR / MPC ←—  
/quadcopter/odom ↓ /quadcopter/command/wrench ↓ motor_allocator →  
/quadcopter/command/rotor_speed ↓ model silników i wirników w Gazebo ↓ siła, momenty, ruch  
modelu → odometria → regulator
```

Element	Rola	Najważniejsze dane
Plik misji	Definiuje waypointy i czasy ich osiągnięcia	t, x, y, z, yaw
Generator trajektorii	Interpoluje waypointy i publikuje pozycję zadaną	PoseStamped
Regulator	Minimalizuje błąd położenia i orientacji	Wymagany ciąg i momenty
Alokator	Rozdziela sterowanie na cztery wirniki	ω_{FL} , ω_{FR} , ω_{RL} , ω_{RR}
Model silników	Modeluje opóźnienie i aerodynamikę wirników	$F_i = k_f \omega_i^2$, $M_i = k_m \omega_i^2$
Gazebo	Oblicza dynamikę i kolizje	Pozycja, orientacja, prędkości
RViz	Porównuje trasę zadaną z rzeczywistą	nav_msgs/Path

3. Świat Gazebo i model quadcoptera

Pliki empty.world i obstacle_course.world opisują podłoże, oświetlenie, przeszkody i parametry fizyki. Krok symulacji wynosi 0,002 s, czyli silnik fizyczny może wykonywać 500 aktualizacji na sekundę.

Model drona ma masę 1,5 kg i momenty bezwładności:

Parametr	Wartość	Znaczenie
m	1,5 kg	Masa całkowita
I _{xx}	0,030 kg·m ²	Bezwładność dla roll
I _{yy}	0,030 kg·m ²	Bezwładność dla pitch
I _{zz}	0,055 kg·m ²	Bezwładność dla yaw
ramię X/Y	0,23 m	Odległość wirnika od środka

Plugin gazebo_actuator jest częścią modelu SDF. Odczytuje położenie i prędkości bryły, publikuje odometrię oraz wyznacza siłę i momenty pochodzące od czterech wirników.

4. Misja i generator trajektorii

Misja jest listą waypointów. Przykładowy wpis:

```
- {t: 18.0, x: 4.0, y: 1.3, z: 1.8, yaw: 0.0}
```

Oznacza, że w chwili 18 s pozycja referencyjna powinna wynosić (4,0; 1,3; 1,8) m, a zadany yaw 0 rad. Pomiędzy waypointami wykorzystywana jest funkcja smoothstep:

$$s(u) = u^2(3 - 2u), 0 \leq u \leq 1$$

Pozycja zadana jest następnie obliczana jako:

$$p(t) = p_0 + s(u)(p_1 - p_0)$$

Dzięki temu prędkość zadana nie zmienia się skokowo na początku i końcu segmentu. Węzeł pracuje z okresem 0,02 s, czyli publikuje pozycję zadaną nominalnie z częstotliwością 50 Hz.

Temat ROS 2	Typ	Przeznaczenie
/quadcopter/command/pose	geometry_msgs/PoseStamped	Aktualna pozycja i yaw zadane
/quadcopter/path/reference	nav_msgs/Path	Kompletna trasa referencyjna
/quadcopter/path/actual	nav_msgs/Path	Historia rzeczywistego ruchu

5. Regulatory lotu

5.1 Kaskadowy PID

PID jest regulatorem domyślnym. Zewnętrzna pętla oblicza błędy położenia:

$$e_x = x_{\text{zad}} - x, e_y = y_{\text{zad}} - y, e_z = z_{\text{zad}} - z$$

Dla osi poziomych wyznaczone są żądane przyspieszenia. Następnie przeliczane są one na zadane kąty roll i pitch. Ruch poziomy powstaje przez przechylenie drona: wektor ciągu, normalnie skierowany pionowo względem kadłuba, uzyskuje składową poziomą.

Wewnętrzna pętla stabilizuje roll, pitch i yaw. Człon proporcjonalny reaguje na aktualny błąd, całkujący usuwa długotrwały uchyb, a różniczkujący tłumi ruch za pomocą informacji o prędkości.

$$u = K_p \cdot e + K_i \int e \, dt - K_d \cdot v$$

Regulator dodaje składnik równoważący ciężar $m \cdot g$ i kompensuje utratę pionowej składowej ciągu podczas przechylenia.

5.2 LQR

LQR wykorzystuje liniowy model dynamiki w otoczeniu zawisu. Sterowanie ma postać:

$$u = -K(x - x_{\text{ref}})$$

Wektor stanu zawiera błędy położenia, prędkości, kąty i prędkości kątowe. Wzmocnienia K dobierają kompromis pomiędzy szybkim zmniejszaniem błędu a intensywnością sterowania. W tej implementacji są zapisane bezpośrednio w `lqr_controller.yaml`.

5.3 MPC

MPC korzysta z dyskretnego modelu ruchu i problemu sterowania optymalnego o skończonym horyzoncie. Domyślny horyzont obejmuje 35 kroków po 0,04 s, czyli około 1,4 s przewidywania. Funkcja kosztu karze błędy stanu oraz duże sterowania. Do modelu wysyłane jest pierwsze sterowanie z rozwiązania, po czym obliczenia są powtarzane dla nowego stanu.

Wspólny interfejs regulatorów: każdy regulator odbiera tę samą pozycję zadaną i odometrię oraz publikuje wymagany całkowity ciąg i momenty. Dzięki temu model silników nie zależy od wybranej metody regulacji.

6. Alokacja sterowania na wirniki

Regulator publikuje wirtualne polecenie Wrench :

- T — całkowity ciąg w niutonach,
- τ_x — moment roll w $N \cdot m$,
- τ_y — moment pitch w $N \cdot m$,
- τ_z — moment yaw w $N \cdot m$.

Alokator rozwiązuje odwrotne zadanie mieszania. Najpierw wyznacza wymagany ciąg każdego wirnika, a następnie jego prędkość:

$$\omega_i = \sqrt{(\max(0, F_i) / k_f)}$$

Indeks	Wirnik	Kierunek	Wpływ na moment
0	Front-left (FL)	CCW	+roll, -pitch, +yaw
1	Front-right (FR)	CW	-roll, -pitch, -yaw

Indeks	Wirnik	Kierunek	Wpływ na moment
2	Rear-left (RL)	CW	+roll, +pitch, –yaw
3	Rear-right (RR)	CCW	–roll, +pitch, +yaw

Prędkości są ograniczane do przedziału 0–1100 rad/s. Jeśli wymagany ciąg pojedynczego wirnika wychodzi ujemny, zostaje ustawiony na zero. Jest to prosta alokacja bez zaawansowanego mechanizmu priorytetów i desaturacji.

7. Model silników i wirników

Zadana prędkość nie jest osiągana natychmiast. Dynamikę silnika opisuje człon pierwszego rzędu:

$$d\omega/dt = (\omega_{\text{zad}} - \omega) / \tau_m$$

Stała czasowa τ_m wynosi 0,06 s. W każdej iteracji Gazebo aktualizuje prędkość wirnika, a następnie oblicza:

$$F_i = k_f \cdot \omega_i^2, M_i = k_m \cdot \omega_i^2$$

Parametr	Wartość	Jednostka
k_f	$8,5 \cdot 10^{-6}$	N/(rad/s) ²
k_m	$1,6 \cdot 10^{-7}$	N·m/(rad/s) ²
τ_m	0,06	s
ω_{max}	1100	rad/s

Dla poziomego zawisu wymagany jest ciąg równy ciężarowi:

$$T = m \cdot g = 1,5 \cdot 9,81 \approx 14,72 \text{ N}$$

Każdy z czterech wirników wytwarza wtedy około 3,68 N, co odpowiada prędkości około 658 rad/s, czyli 6280 obr./min. Maksymalny łączny ciąg modelu wynosi około 41,14 N.

8. Pełna pętla sprzężenia zwrotnego

- 1 Generator trajektorii oblicza pozycję zadaną dla bieżącego czasu.
- 2 Gazebo publikuje aktualny stan drona jako odometrię.
- 3 Regulator oblicza błędy oraz wymagany ciąg i momenty.
- 4 Alokator przelicza je na cztery zadane prędkości wirników.
- 5 Model silników stopniowo zbliża rzeczywiste prędkości do zadanych.
- 6 Model wirników wyznacza ciągi i momenty aerodynamiczne.
- 7 Gazebo całkuje równania ruchu i otrzymuje nowy stan.
- 8 Nowy stan wraca do regulatora; cykl jest powtarzany.

Jest to układ zamknięty. Zakłócenie, błąd modelu lub opóźnienie silników powodują odchylenie od trajektorii, które regulator próbuje skorygować w następnych iteracjach.

9. Gazebo i RViz

Gazebo przedstawia fizyczny świat, przeszkody i ruch modelu. Turkusowa linia dodana do świata jest bezkolizyjną wizualizacją trajektorii referencyjnej. Nie generuje sił ani kontaktów.

RViz pokazuje dokładne dane ROS 2:

- turkusowa linia — pełna trasa referencyjna,
- czerwona linia — rzeczywista trasa odczytana z odometrii,
- siatka 50×50 m — odniesienie przestrzenne.

Różnica między liniami obrazuje uchyb śledzenia. Zaokrąglenia czerwonej linii w pobliżu zmian kierunku wynikają z bezwładności, ograniczeń sterowania, dynamiki silników i nastaw regulatora.

10. Uruchamianie i diagnostyka

Budowanie i uruchomienie

```
cd /home/marton/Repos/drone_ws
colcon build --packages-select quadcopter_sim
source install/setup.bash
ros2 launch quadcopter_sim simulation.launch.py \
  scenario:=obstacle_course controller:=pid
```

Wybór regulatora

```
# PID
controller:=pid
```

```
# LQR
controller:=lqr
```

```
# MPC
controller:=mpc
```

Najważniejsze tematy

Polecenie	Obserwowana wielkość
ros2 topic echo /quadcopter/command/pose	Pozycja zadana
ros2 topic echo /quadcopter/odom	Pozycja, orientacja i prędkości rzeczywiste
ros2 topic echo /quadcopter/command/wrench	Wymagany ciąg i momenty regulatora
ros2 topic echo /quadcopter/command/rotor_speed	Zadane prędkości czterech wirników
ros2 topic echo /quadcopter/rotor_speed	Rzeczywiste prędkości po dynamice silników

Interpretacja diagnostyki

- W zawisie wszystkie prędkości powinny być zbliżone do 658 rad/s.
- Różnica lewa-prawa powoduje roll.
- Różnica przód-tył powoduje pitch.
- Różnica par CW-CCW powoduje yaw.
- Długie utrzymywanie 0 lub 1100 rad/s oznacza nasycenie alokatora.
- Duża różnica pomiędzy prędkością zadaną i rzeczywistą wskazuje na szybką zmianę polecenia lub opóźnienie silnika.

11. Założenia i ograniczenia

- Model ciągu jest kwadratowy i nie uwzględnia przepływu postępowego ani efektu podłoża.
- Nie jest modelowana aerodynamika śmigła zależna od jego geometrii.
- Nie ma modelu elektrycznego akumulatora, ESC, napięcia, prądu i spadku napięcia.
- Prędkość wirnika jest sterowaniem wykonawczym; napięcie nie jest jeszcze sygnałem wejściowym.
- Siły czterech wirników są sumowane i przykładane do bryły jako równoważna siła oraz momenty.
- Alokator stosuje proste obcięcie wartości zamiast optymalnej alokacji przy nasyceniu.

- Regulatory korzystają z idealnej odometrii Gazebo bez szumu czujników i estymatora stanu.

Możliwy następny etap: model elektryczny może przekształcać napięcie PWM/ESC na prąd i moment silnika, uwzględniając rezystancję, stałą SEM, bezwładność wirnika oraz napięcie akumulatora. Wtedy bezpośrednim sygnałem sterującym byłoby napięcie lub wypełnienie PWM.

12. Pliki źródłowe

Plik	Odpowiedzialność
launch/simulation.launch.py	Uruchamianie świata, misji, regulatora, alokatora i RViz
scripts/trajectory_node.py	Interpolacja misji i publikowanie ścieżek
src/quadcopter_controller.cpp	Kaskadowy PID
src/lqr_controller.cpp	Regulator LQR
src/mpc_controller.cpp	Regulator MPC
src/motor_allocator.cpp	Mieszanie ciągu i momentów na prędkości wirników
src/gazebo_actuator.cpp	Dynamika silników, siły wirników i odometria
models/quadcopter/model.sdf	Masa, bezwładność, geometria i parametry napędu
config/*.yaml	Nastawy regulatorów, misje i model silników
worlds/*.world	Światy i przeszkody Gazebo

Dokument opisuje stan implementacji pakietu quadcopter_sim z dnia 28.08.2026.